

课程设计总结:

1. 电路原理图设计心得体会

在设计 16×16 点阵控制系统电路原理图的过程中,对单片机的工作原理有了更深入的理解。单片机作为核心控制单元,通过 I/O 口与点阵显示屏连接,实现对显示内容的控制。掌握了点阵显示屏的驱动原理,了解到如何通过行列扫描的方式来点亮特定的 LED 点,从而形成所需的字符或图案。在绘制电路原理图时,布线是一个关键问题。由于 16×16 点阵显示屏的引脚较多,如何合理布线以避免线路交叉和混乱是一大挑战。解决方法是采用分层设计和模块化设计的思想。将电源线路、控制信号线路和数据传输线路分开设计,同时将点阵显示屏的驱动电路设计成独立的模块,便于调试和修改。

2. C51 程序设计心得体会

在程序调试中,经常遇到逻辑错误,如显示图案错误或无法正常显示等。通过仔细检查代码逻辑、分析变量取值和程序流程,逐步找出并解决问题,这极大地提高了逻辑分析能力。硬件电路的问题也会导致程序运行异常,因此需要将硬件调试与软件调试相结合。学会了使用示波器等工具来检测信号,排查硬件连接和电路故障,同时根据硬件的实际情况来调整软件程序。

3. 仿真心得体会

在仿真过程中,经常会遇到点阵显示异常的情况,如显示内容不全、显示亮度不均匀等。这可能是由于扫描算法不合理或程序中数据处理不当导致的。通过仔细分析点阵的扫描原理,对程序中的行列扫描代码进行优化,例如调整扫描间隔时间和数据发送顺序,解决了显示异常问题。仿真经验为今后进行更复杂的单片机系统设计奠定了基础。可以将在 16×16 点阵控制系统设计仿真中积累的电路设计和程序调试方法应用到其他系统中,更大规模的点阵显示系统或多功能单片机控制系统。

4. 程序下载心得体会

时常遇到程序无法下载的困境,原因多样。有时是电源问题,开发板供电不足导致下载中断,需外接稳定电源;有时是因为之前下载错误程序,单片机进入死锁状态,利用复位键配合重新设置下载选项得以解决。

目 录

1 概述	1
1.1 课程设计思路	1
1.1.1 问题分析	1
1.1.2 方案设计	1
1.1.1 创新点	2
1.2 课程设计内容和要求	2
1.2.1 系统功能模块划分	2
1.2.2 各模块的实现方案	3
1.2.3 可以量化的性能指标	3
1.2.4 输入/输出接口说明	4
1.3 开发环境	4
1.3.1 软件环境	4
1.3.2 硬件环境	4
1.3.3 辅助工具	4
2 硬件设计	5
2.1 硬件原理框图	5
2.2 最小系统电路设计	5
2.2.1 电源、复位电路设计及说明	5
2.2.2 时钟电路设计及说明	6
2.2.3 按键电路设计及说明	6
2.2.4 显示电路设计及说明	6
2.3 功能模块电路设计	7
2.3.1 控制单元模块电路设计	7
2.2 行驱动单元模块电路设计	8
2.3 列驱动单元模块电路设计	9

3 软件设计	10
3.1 系统软件总体设计	10
3.2 主程序流程图	10
3.3 关键算法实现	12
3.4 子程序设计说明	12
3.4.1 LED 点阵显示子程序设计	12
3.4.2 计时子程序设计	13
3.4.2 键盘扫描子程序设计	14
3.4.4 子程序接口设计说明	15
4 仿真与调试	错误！未定义书签。
4.1 取字模设置	16
4.2 仿真测试	17
4.2 实验箱测试	19
参考文献	21
附录 1 PROTEUS 仿真电路图	22
附录 2 关键部分程序清单	24

1 概述

1.1 课程设计思路

1.1.1 问题分析

随着信息显示技术的不断发展，LED 点阵显示屏因其结构简洁、功耗低、显示直观而被广泛应用于公共信息发布、仪表显示及智能终端等领域。然而，小型 LED 点阵显示系统普遍存在功能单一、显示效果有限、交互性不足等问题。传统系统多采用固定字符或静态画面显示，难以实现文字滚动与动态切换，且用户控制方式较为原始，不利于实际应用拓展。

本课程设计旨在针对上述问题，构建一个基于单片机的 16×16 LED 点阵汉字及图文显示系统。该系统需在保证硬件结构简洁的前提下，实现汉字动态滚动显示、图文切换及按键交互控制等功能，从而提升系统的显示效果与可操作性，为后续的多功能显示系统设计提供参考。

1.1.2 方案设计

本系统以 AT89C51 单片机 为核心控制器，采用 行列动态扫描方式 驱动 16×16 LED 点阵模块。整体设计分为硬件结构与软件流程两部分。

1. 硬件设计部分：

单片机最小系统 提供时钟与复位电路，保证系统稳定运行；

- 点阵驱动电路 由 74HC595 移位寄存器和 74HC138 译码器组成，74HC595 负责列信号的串行输入与并行输出，74HC138 实现行选通控制，两者协同完成整屏点阵的逐行动态扫描；
- 按键输入模块 连接至单片机 I/O 口，用于实现显示内容、滚动方向及显示模式的实时切换，增强系统的人机交互能力。

2. 软件设计部分：

系统软件采用模块化设计思路，主要包括四个功能模块：

- 字模存储模块：将常用汉字及图案的点阵数据以二维数组形式存储于程序存储区；
- 显示驱动模块：通过定时器中断周期性刷新行扫描信号，实现 LED 点

阵的稳定显示；

- 滚动控制模块：根据设定方向与速度，实现文字的左右或上下滚动；
- 按键检测模块：监测用户输入并根据状态切换显示内容与滚动方式。
- 整个系统通过中断机制实现显示与输入的并行处理，从而保证画面流畅、响应灵敏。

1.1.1 创新点

1. 模块化结构设计：程序按功能分层，结构清晰、耦合度低，便于后续维护与功能扩展；
2. 多模式滚动显示：实现文字在横向与纵向多方向滚动显示，支持多字符连续移动，视觉效果更丰富；
3. 交互式控制机制：用户可通过按键切换显示内容、滚动方向与速率，增强系统的可玩性与灵活性；
4. 高效动态刷新策略：利用定时中断和移位寄存器协同控制，提高显示亮度与刷新稳定性，减少闪烁现象；
5. 资源优化与扩展性强：在有限的硬件资源条件下实现多功能显示，可扩展为多屏联动或图像显示系统。

1.2 课程设计内容和要求

1.2.1 系统功能模块划分

本系统基于 AT89C51 单片机设计，实现 16×16 LED 点阵的汉字及图文动态显示。整体功能模块主要包括以下四部分：

1. 主控单元模块：由 AT89C51 单片机构成，负责系统的总体控制，包括行列扫描信号输出、数据刷新及按键检测；
2. 显示驱动模块：由 74HC595 移位寄存器与 74HC138 译码器组成，用于完成列数据的移位输出与行信号的译码选通，实现 LED 点阵的动态扫描显示；
3. 按键输入模块：设置“启动/停止”“方向”“加速”“减速”四个按键，实现显示内容的启停控制、滚动方向调整及显示速度调节；

4. 显示单元模块：由 16×16 LED 点阵屏构成，用于显示汉字或图形信息，通过扫描方式实现点亮控制。

1.2.2 各模块的实现方案

1. 主控单元模块实现方案：

系统采用 AT89C52 单片机作为核心控制器，利用其丰富的 I/O 接口实现数据传输与外设控制。单片机内部定时器用于产生稳定的中断信号，以控制扫描时序与刷新频率，从而保证显示稳定不闪烁。

2. 显示驱动模块实现方案：

列驱动部分使用两个级联的 74HC595 移位寄存器，将单片机的串行输出数据转换为并行信号，从而简化接口并减少占用的 I/O 端口。行驱动部分由两个 74HC138 三选八译码器组成，实现对 16 行的分时选通。通过单片机输出的行选信号与列数据的同步控制，实现整个 16×16 点阵的动态刷新显示。

3. 按键输入模块实现方案：

四个按键分别连接至单片机 I/O 端口，通过软件轮询或中断方式实现按键状态检测。当检测到按键动作时，程序根据按键功能调整系统参数，如暂停滚动、切换方向或改变刷新速度。

4. 显示单元模块实现方案：

显示模块采用 16×16 LED 点阵器件，行线与列线分别与 74HC138 和 74HC595 相连。系统按行扫描方式工作，每次点亮一行的对应列数据，通过快速刷新实现整屏汉字或图形显示。字模数据以数组形式存储在程序存储器中，通过循环调用实现多字滚动与连续显示。

1.2.3 可以量化的性能指标

1. 显示分辨率： 16×16 点阵（256 像素）；
2. 显示刷新频率： ≥ 60 Hz，显示稳定无闪烁；
3. 汉字滚动速度调节范围：支持 3~5 档速度调节；
4. 响应时间：按键输入响应时间 ≤ 50 ms；
5. 系统稳定性：连续运行 1 小时无显示错乱或停顿现象；
6. 显示内容容量：可存储至少 8 个汉字或图形数据并循环显示。

1.2.4 输入/输出接口说明

1. 输入接口：
 - 按键输入端：共 4 个按键，分别对应“启动/停止”“方向切换”“加速”“减速”；
 - 电源输入端：+5V 直流供电。
2. 输出接口：
 - 行选信号输出端：由 74HC138 译码输出，控制 LED 点阵行选通；
 - 列数据信号输出端：由 74HC595 并行输出，控制列点亮状态；
 - 显示输出端：连接 16×16 LED 点阵模块，实现最终显示效果。

1.3 开发环境

1.3.1 软件环境

IDE: Keil μ Vision 5 (C51 编译器)

仿真工具: Proteus 8 Professional (含 MCU 模型及外设库)

1.3.2 硬件环境

主控芯片: Intel 8051AH (11.0592MHz 晶振)

外设模块: 如 16×16LED 点阵, 独立按键

1.3.3 辅助工具

PCtoLCD2002

2 硬件设计

2.1 硬件原理框图

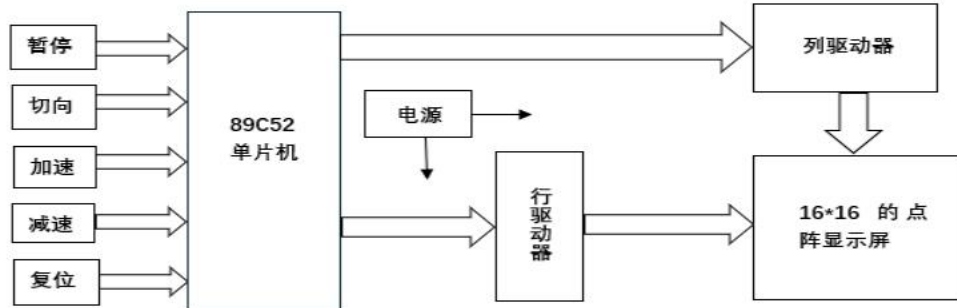


图 2.1 基于单片机的 16x16 点阵控制系统方框图

当打开电源后，单片机自动启动，执行 ROM 中的程序，单片机的 P0 和 P2 口进行对点阵屏的行驱动，74HC138 对点阵屏的列进行驱动。

2.2 最小系统电路设计

2.2.1 电源、复位电路设计及说明

电源 POWER 部分，通过一个开关（SW1）控制电源的通断。一个标记为“复位”的按钮（S1），用于手动复位单片机。

电源电路设计图和复位电路设计图分别如图 2.2 和图 2.3 所示。

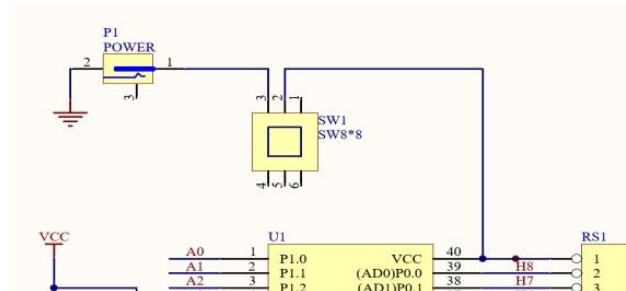


图 2.2 电源电路设计图

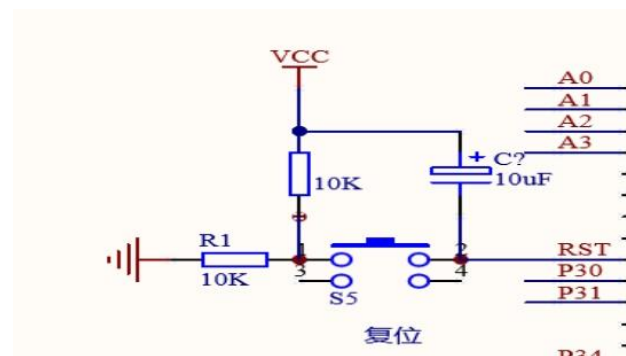


图 2.3 复位电路设计图

2.2.2 时钟电路设计及说明

时钟电路通常由晶振和两个电容组成，用于生成单片机的时钟信号。晶振的两端分别连接到单片机的 XTAL1 和 XTAL2 引脚，电容的一端接地，另一端分别连接到 XTAL1 和 XTAL2 引脚。时钟电路是单片机系统的核心，其设计和实现直接影响到整个系统的稳定性和性能。

在设计时钟电路时，需要选择合适的晶振和电容，确保时钟信号的稳定性和准确性。通过合理的设计和调试，可以为单片机提供一个可靠的时钟源，从而保证 16×16 点阵控制系统的正常运行，时钟电路设计图如图 2.4 所示。

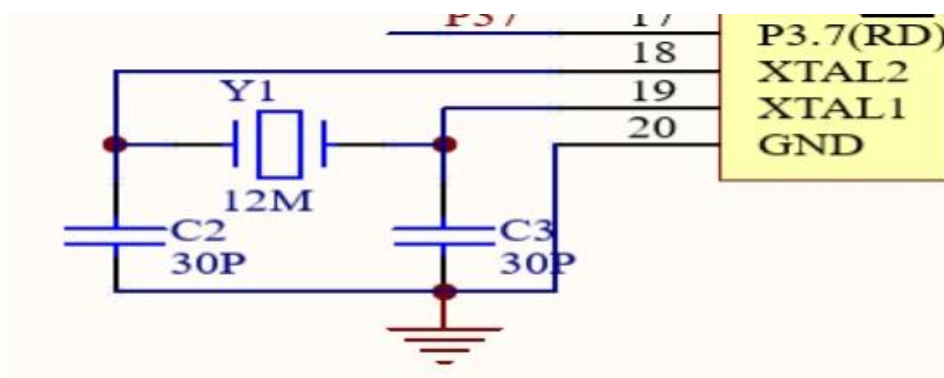


图 2.4 时钟电路设计图

2.2.3 按键电路设计及说明

有四个标记为“切换”、“暂停”、“加速”、“减速”的按键 (S2-S5)。这些按键用于用户输入。每个按键的一端连接到单片机的 I/O 引脚，另一端通过上拉电阻连接到电源 (VCC)。这样，在按键未被按下时，I/O 引脚处于高电平状态。按键电路设计图如图 2.5 所示。

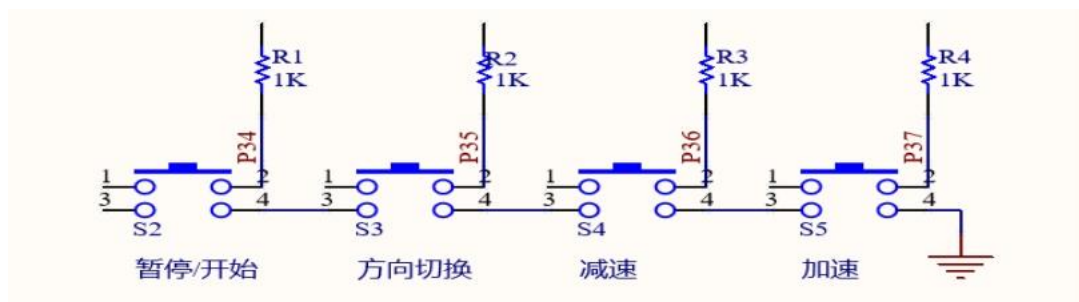


图 2.5 按键电路设计图

2.2.4 显示电路设计及说明

显示电路主要由 16×16 点阵 LED 模块、驱动电路和控制电路组成。显示电路的核心是 16×16 点阵 LED 模块，它由 16 行和 16 列的 LED 组成，总共 256

个 LED。这个模块被分成了四个 8×8 的点阵模块（R81, R82, R83, R84），每个模块控制 16 个 LED 的行或列。显示电路设计图如图 2.6 所示。

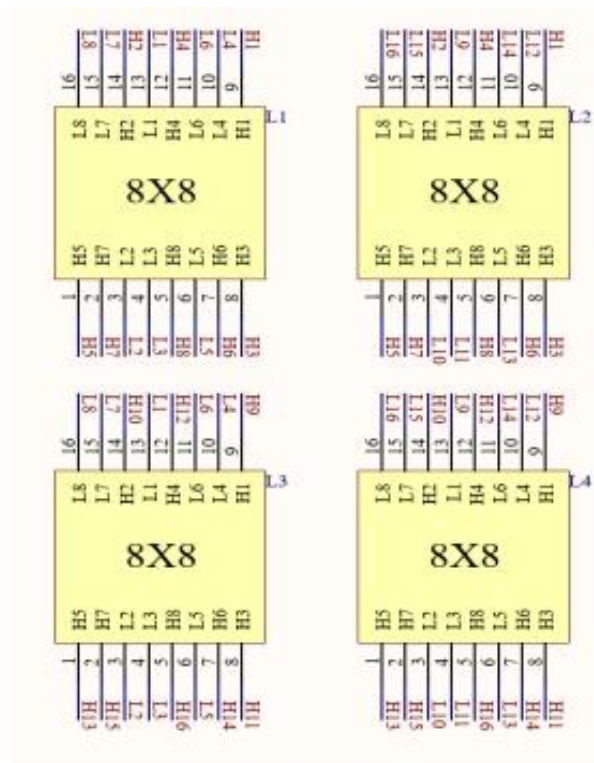


图 2.6 显示电路设计图

2.3 功能模块电路设计

2.3.1 控制单元模块电路设计

控制单元以 AT89C51 单片机为核心，构建最小系统电路实现时钟、复位与 I/O 控制功能。晶振部分采用 12MHz 石英晶体 X1 及两个 22pF 电容 C1、C2 接地构成振荡电路，为单片机提供稳定的时钟信号，保证程序按节拍顺序执行。

复位部分由电容 C3（1 μ F）、电阻 R1（10 Ω ）及复位按键组成，上电时电容充电使 RST 引脚保持高电平，单片机完成初始化；手动按下按键时可强制系统重新启动。单片机通过 P0、P1、P2、P3 口与外围模块相连，完成 LED 点阵驱动、按键检测及通信等任务。

整体电路设计结构简洁、运行稳定，为系统提供可靠的控制核心，设计仿真电路如图 2.7 所示。

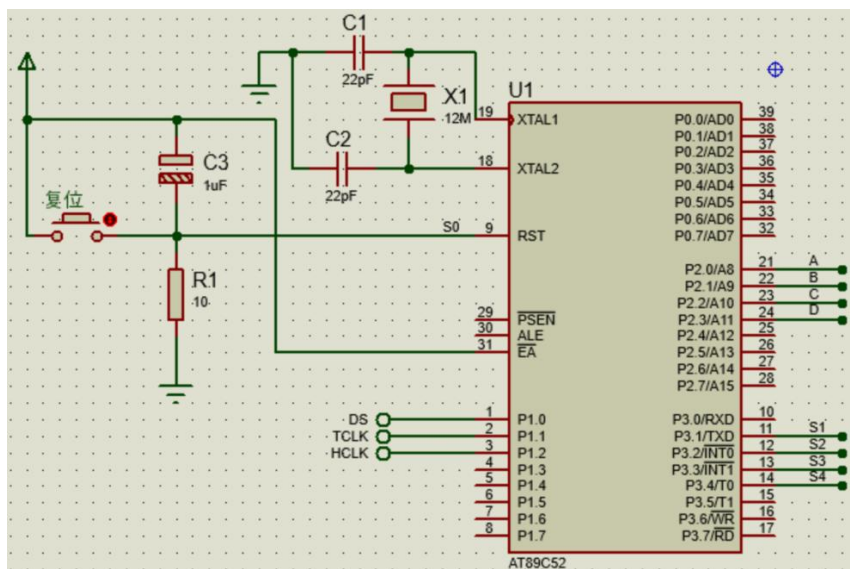


图 2.7 控制单元模块仿真设计

2.2 行驱动单元模块电路设计

行驱动模块采用两片 74HC595 串入并出移位寄存器，通过级联实现对 16 行 LED 的独立控制。单片机通过三根控制线（DS、SH_CP、ST_CP）将行数据按位输入 595 内部寄存器，移位时钟驱动数据移位，锁存时钟将数据同步输出至行端。两片 595 级联后输出 16 路并行信号，与 LED 点阵行线相连。

该设计实现了串并转换，减少了单片机 I/O 口占用，同时保证输出信号的稳定性和显示刷新的一致性，为点阵动态扫描提供了基础支持，设计仿真电路如图 2.8 所示。

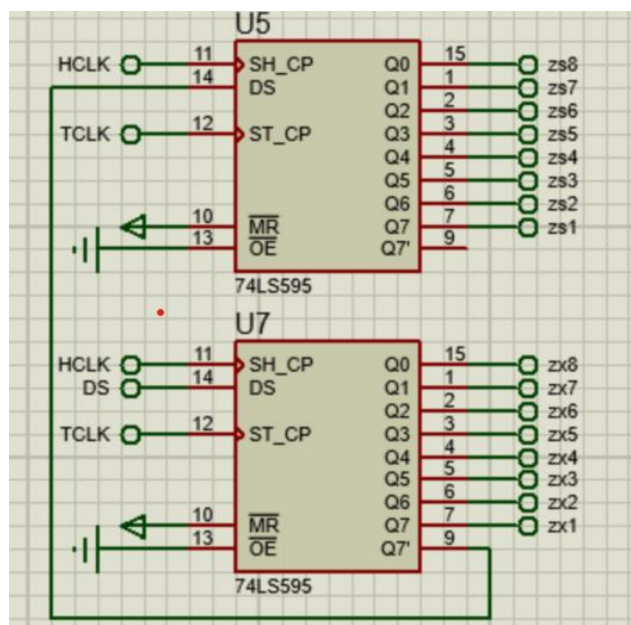


图 2.8 行驱动单元模块仿真设计

2.3 列驱动单元模块电路设计

列驱动模块采用两片 74HC138 三—八线译码器组成 4-16 线译码电路。单片机仅提供 4 位控制信号，即可完成对 16 列的逐列选通。两片 74HC138 的地址输入端并联，共享控制信号，利用高位选择信号实现前 8 列与后 8 列的切换。译码输出端连接 LED 列端，逐列激活显示点。

该设计大幅节省了 I/O 资源，译码方式结构清晰，响应速度快，能有效与行驱动模块配合完成动态显示，实现字符、图形及汉字的稳定输出，设计仿真电路如图 2.9 所示。

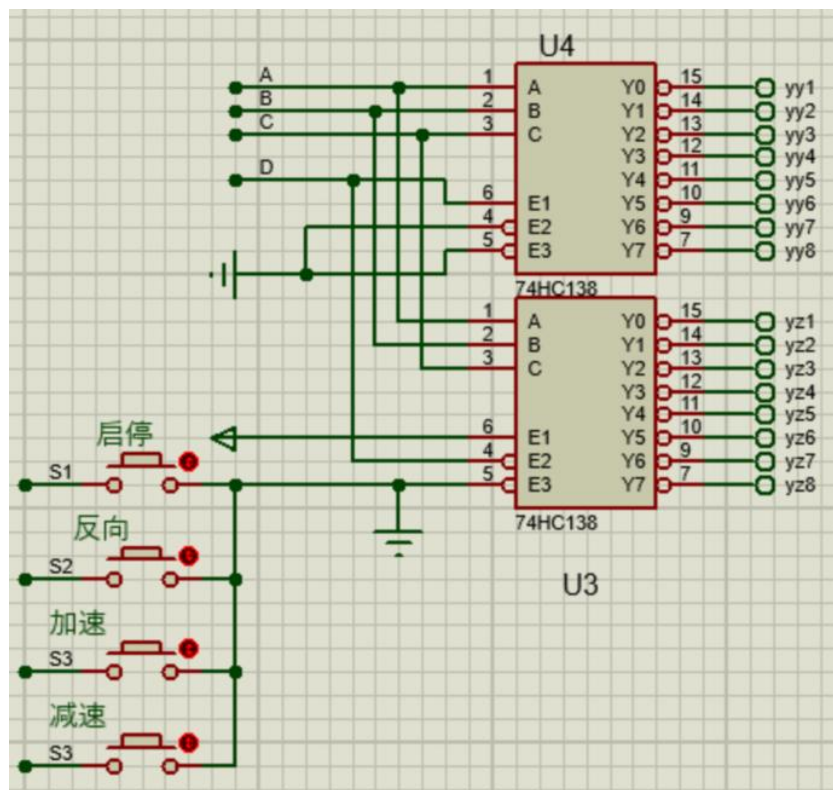


图 2.8 列驱动单元模块仿真设计

3 软件设计

3.1 系统软件总体设计

当单片机供电以后，单片机没有停机指令，显示屏实现汉字的连续左滚动显示效果。当有按键 KEY 按下时，显示屏实现汉字的连续有滚动显示效果。系统程序不断地循环执行上述显示效果。系统软件设计流程框图如图 3.1 所示。

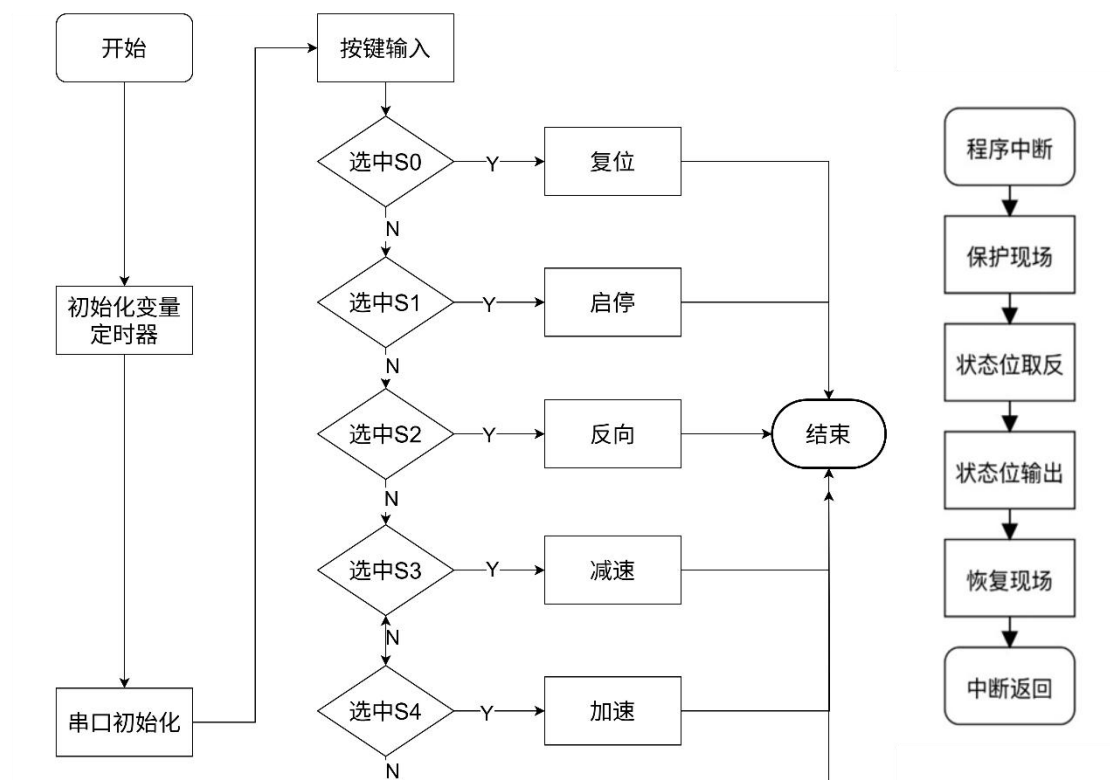


图 3.1 系统软件设计流程框图

3.2 主程序流程图

主程序是系统的核心控制部分，负责完成初始化、任务调度与各子程序的协调运行。程序在系统上电复位后，首先执行硬件初始化，包括 I/O 口方向配置、定时器设置、显示缓存清零 等操作。随后进入主循环，不断完成以下任务：

键盘扫描：检测用户按键输入，判断是否需改变显示状态（启停、方向、速度调整等）；

点阵刷新：将字模数据逐行送入 LED 点阵屏，实现稳定的静态或动态显示；

状态更新：根据中断子程序提供的滚动偏移量 `net`，确定当前显示的起始位

置；

循环执行：以上任务持续运行，实现系统实时响应与流畅显示。

主程序逻辑层次清晰，保证了显示刷新与用户交互的实时性。通过主循环与定时器中断的协作，实现了在有限硬件资源下的流畅汉字滚动与稳定显示。

主程序流程图如图 3.2 所示。

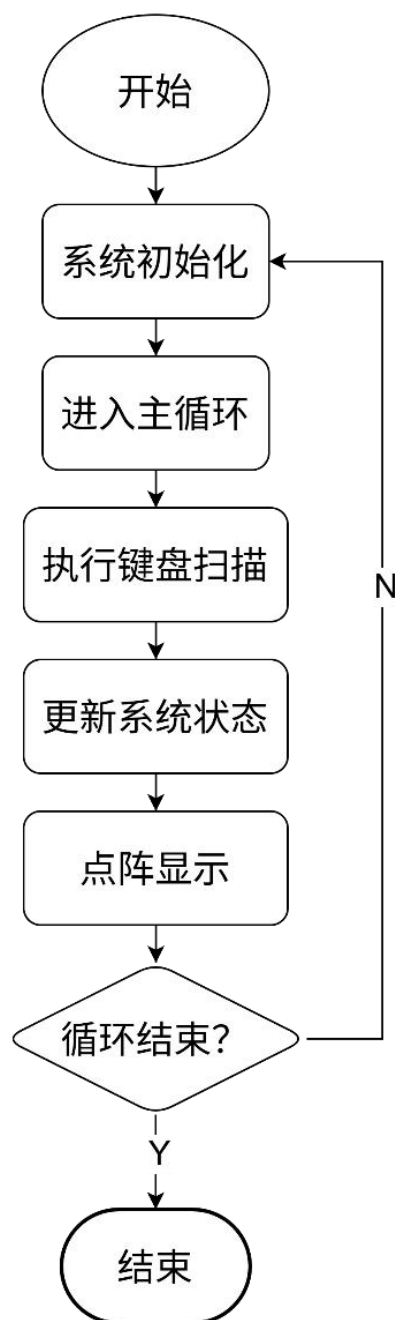


图 3.2 主程序流程图

3.3 关键算法实现

本系统涉及的核心算法为动态扫描显示与滚动偏移控制：

1. 滚动偏移算法伪代码如下所示：

```
if(direction == LEFT)
    net = (net + 1) % MAX_OFFSET;
else
    net = (net == 0) ? MAX_OFFSET - 1 : net - 1;
```

2. 动态扫描算法伪代码如下所示：

```
for(row = 0; row < 16; row++) {
    send_data_to_74HC595(char_bitmap[row + net]);
    select_row_with_74HC138(row);
    delay_ms(1);
}
```

3.4 子程序设计说明

3.4.1 LED 点阵显示子程序设计

点阵显示子程序的主要任务是将存储在字模数组中的 16×16 汉字数据实时输出到 LED 点阵显示屏。采用行列动态扫描方式：

1. 列驱动：利用 74HC595 移位寄存器实现列数据移入与锁存，每次输出 16 位数据对应一行的点阵状态；
2. 行选通：通过 74HC138 译码器选择当前需要点亮的行，其余行保持关闭；
3. 动态刷新：在 10ms 的周期内依次扫描 16 行，当循环足够快时，人眼会感知到完整稳定的汉字显示。

该子程序结构简单但运行频率较高，要求在定时器中断和数据移位上配合良好，以避免出现闪烁或显示畸变，LED 点阵显示子程序流程图如图 3.3 所示。

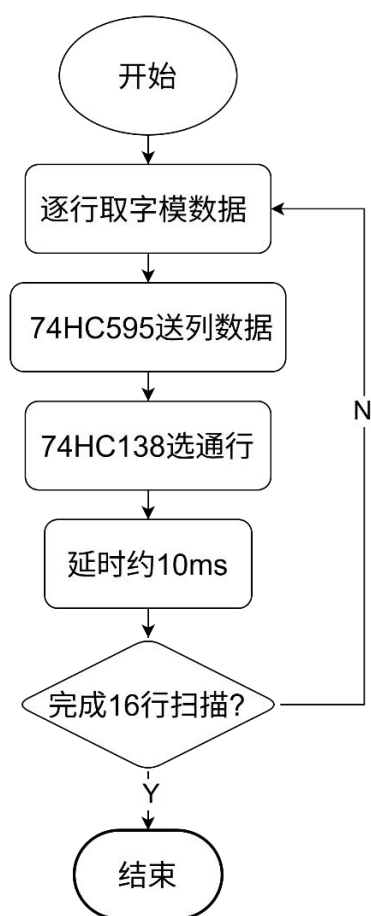


图 3. 3LED 点阵显示子程序流程图

3.4.2 计时子程序设计

定时子程序基于单片机定时器 0 实现，设定定时时间约为 10ms。中断服务函数在每次定时溢出后执行，主要完成以下功能：

1. 时间基准产生：通过定时器溢出周期作为系统的时基；
2. 滚动计数：每次中断对变量 `alt` 自增，当 `alt` 达到预设的 `speed` 值时，触发滚动操作；
3. 偏移量更新：根据滚动方向 `fx` 更新显示偏移量 `net`：
 - 左移时 `net++`，到达末尾时回到起始；
 - 右移时 `net--`，减到 0 时回到末尾；
4. 数据同步：中断更新的偏移量会在主程序刷新点阵显示时被调用，从而完成滚动效果。

该子程序是滚动显示的核心，通过中断方式实现定时与数据偏移更新，保证

了滚动的均匀性与实时性，计时子程序流程图如图 3.5 所示。

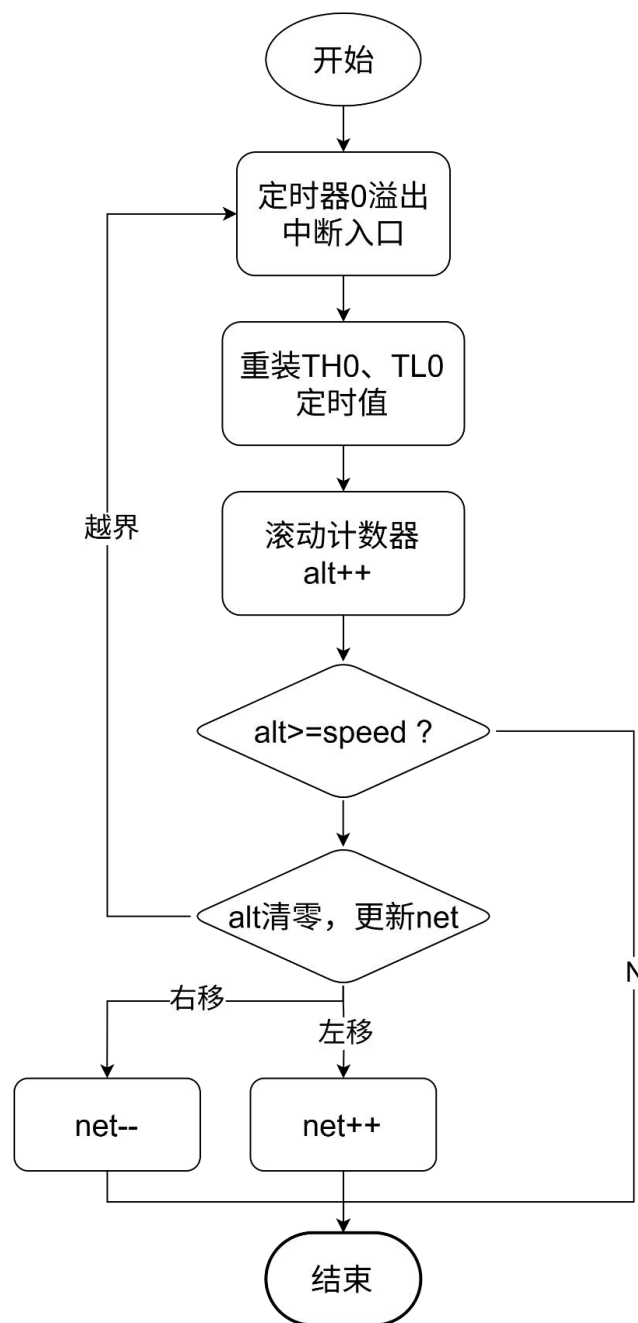


图 3.5 计时子程序流程图

3.4.2 键盘扫描子程序设计

键盘扫描子程序负责用户交互功能的实现。程序通过检测 P3 口输入电平变化来判断按键是否被按下，并结合软件消抖算法（延时或状态判断）确保输入稳定。功能包括：

1. 启动/停止按键：用于控制点阵显示的启停，当停止时保持最后一帧显示；

2. 方向切换按键：在左移与右移模式间切换，使汉字按不同方向滚动显示；
3. 速度加减按键：通过修改变量 speed，从而影响定时器中断中滚动计数的阈值，实现快慢速滚动。

键盘扫描程序的设计直接影响系统的可操作性和交互性，因此在实现中注重响应及时性和抗抖动性能，键盘扫描子程序流程图如图 3.4 所示。

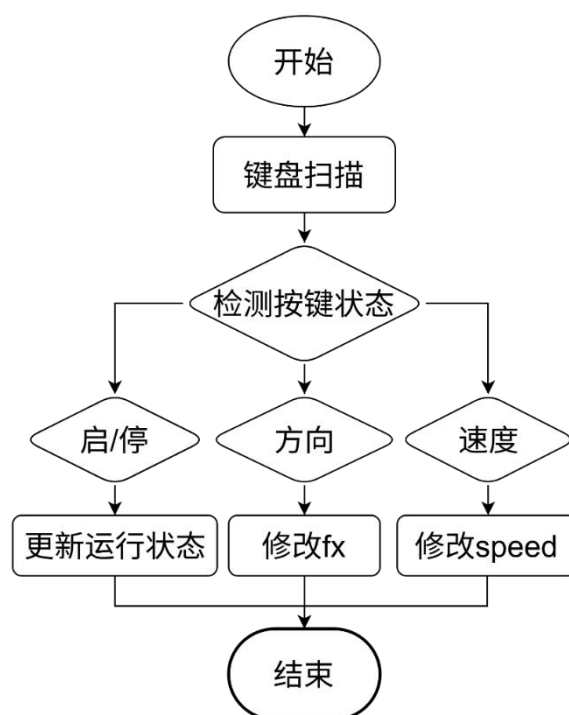


图 3.4 键盘扫描子程序流程图

3.4.4 子程序接口设计说明

上述各个子程序的接口定义与功能描述如表 3.1 所示。

表 2.1 子程序接口

子程序名称	接口定义	功能描述
主程序	无输入/输出	程序入口，初始化系统并循环调度各功能模块
LED点阵显示子程序	无输入/输出	控制字模动态显示，完成列数据输出与行扫描
键盘扫描子程序	无输入/输出	检测按键输入并更新状态变量
计时子程序设计	中断函数，无输入输出	定时10ms更新计时、滚动偏移与刷新标志 初始化 I/O、定时器及中断系统

4 仿真与调试

4.1 取字模设置

打开软件以后，在菜单栏的模式，菜单下选择字符模式，通过界面中上部的这些小工具，可以设置文字的字体，对应字符的大小等信息。通过修改点阵大小下面的工具可以很方便的的对文字进行各种调整，具体配置如图 4.1 所示。

完成对文字的调整以后，还是单击设置图标按钮进行字模设置，具体字模设置如图 4.2 所示。

在图 4.1 中下部的文本框中输入要转换的文字。然后单击生成字模图标按钮，就可以看到文字对应的显示数据了，如需保存可点击保存字模图标按钮，保存为 txt 文件。

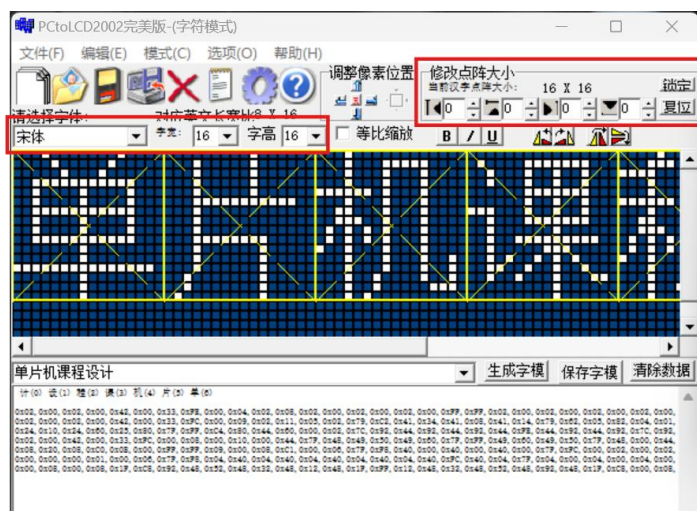


图 4.1 程序初始化仿真结果图

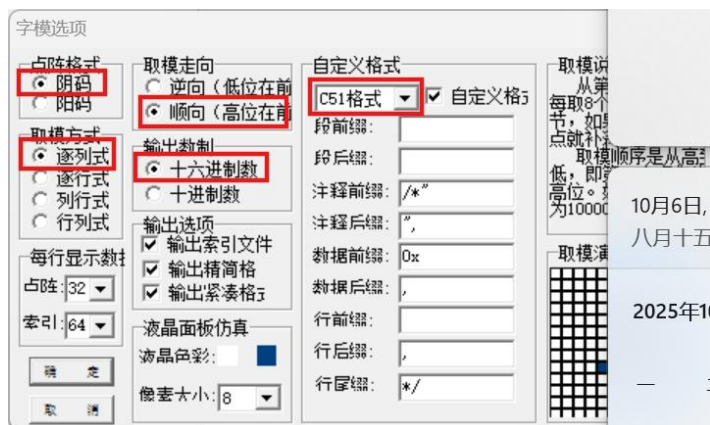


图 4.2 程序初始化仿真结果图

4.2 仿真测试

程序初始化仿真结果如图 4.3 所示，点击暂停仿真结果如图 4.4 所示，点击反向仿真结果如图 4.5 所示，点击复位仿真结果如图 4.6 所示。且图下方有四个按键，第一个是启停，第二个按键是反向，第三个按键是加速，第四个按键是减速。

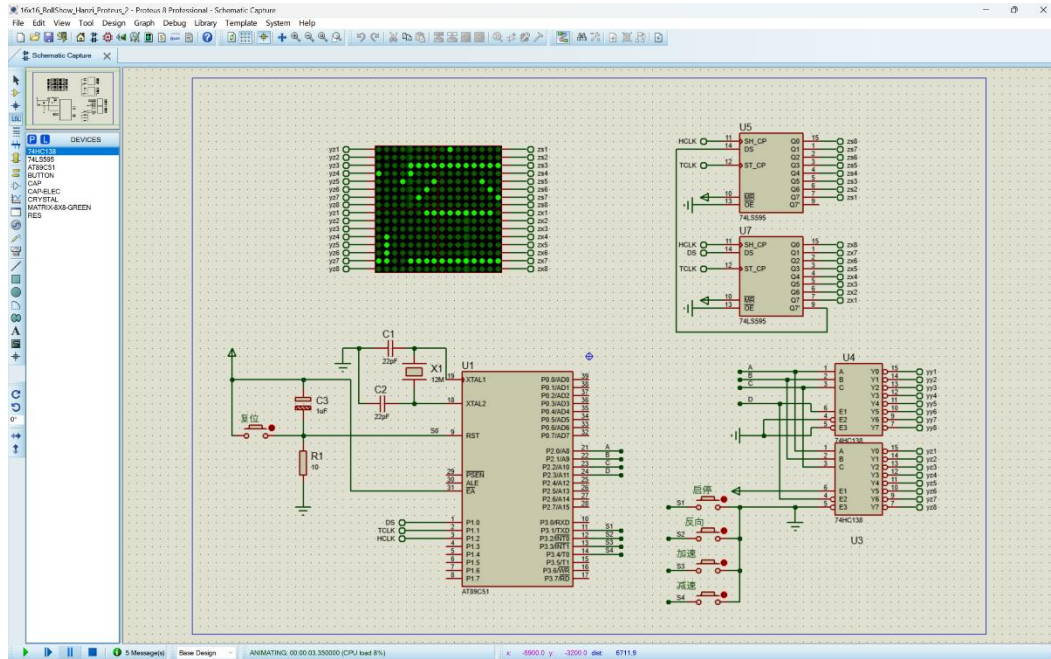


图 4.3 程序初始化仿真结果图

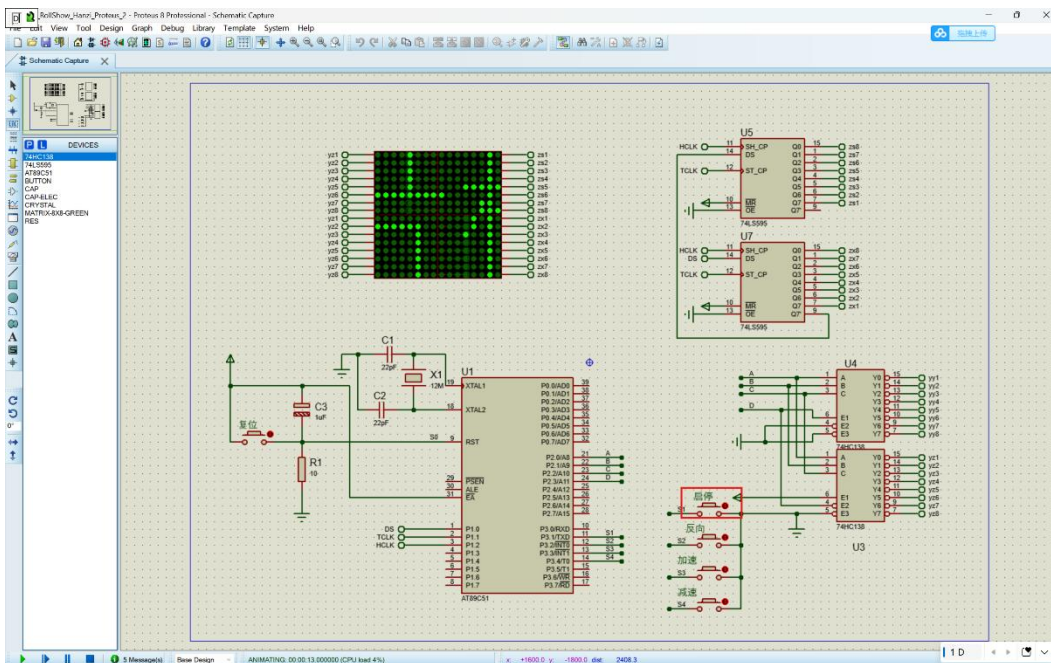


图 4.4 点击暂停仿真结果图

图 4.5 点击反向仿真结果图

图 4.6 点击复位仿真结果图

4.2 实验箱测试

在使用实验箱进行实物调试时,实验箱连线效果如图 4.7 所示, 程序上传效果如图 4.8 所示, 最终调试结果如图 4.9-图 4.10 所示, 且所设计的全部功能按键均可正常实现。

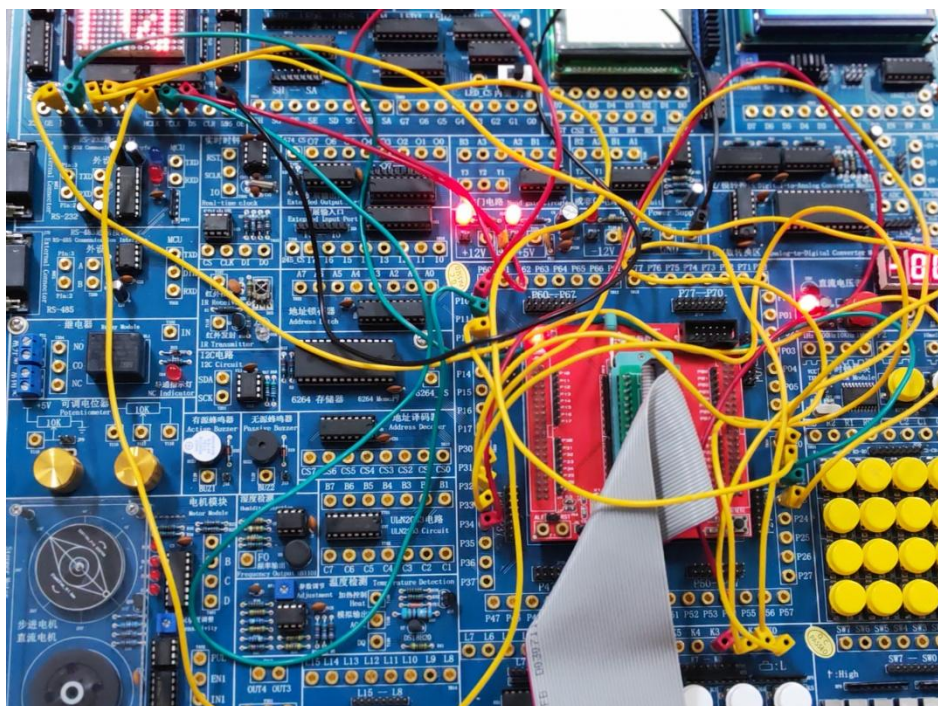


图 4.7 实验箱连线效果图

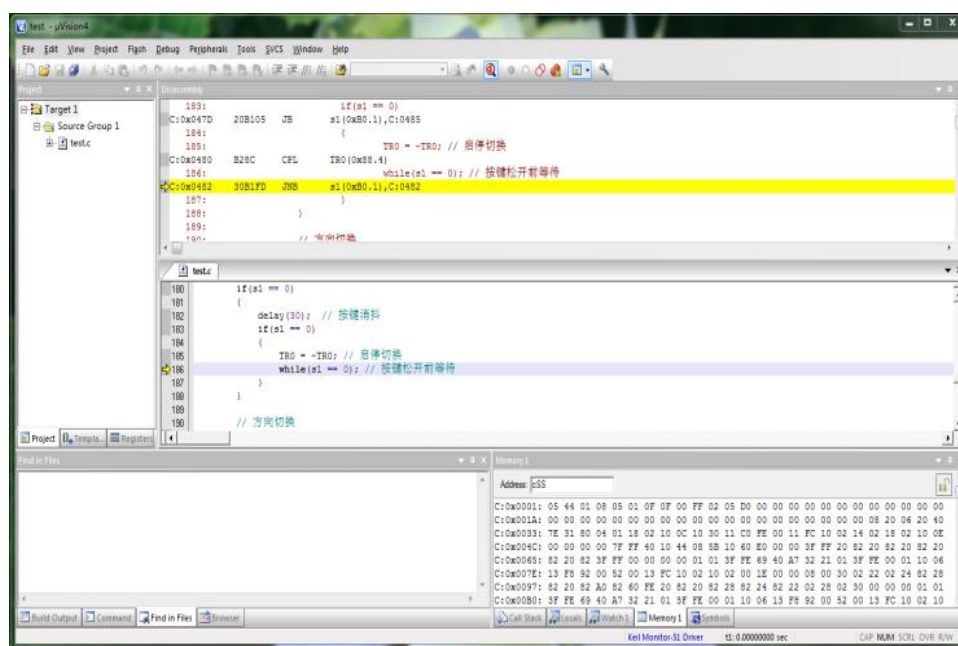


图 4.8 程序上传效果图

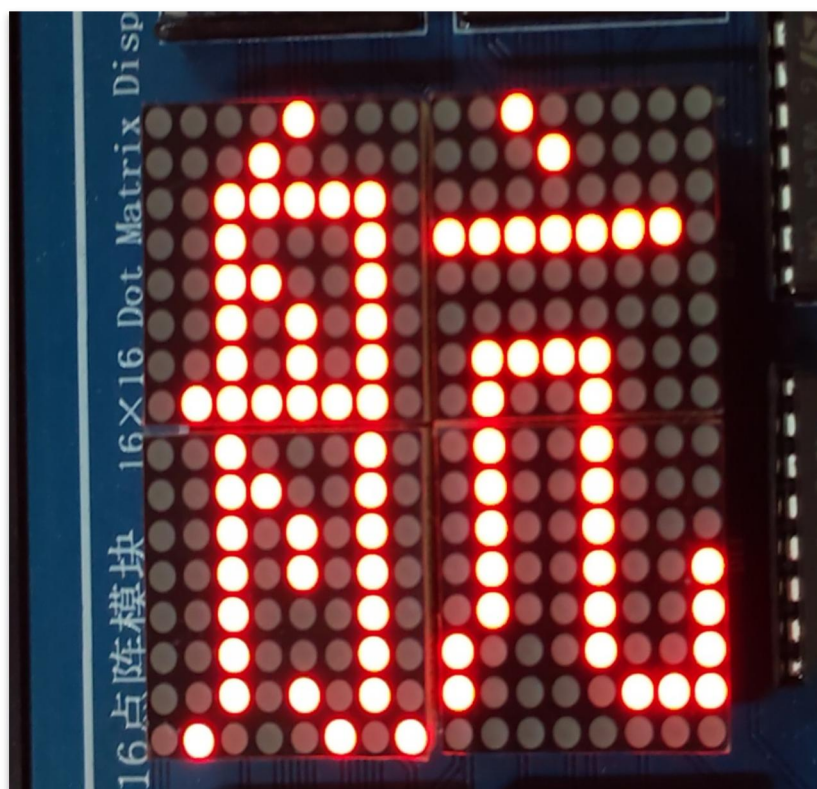


图 4.9 程序初始化最终调试结果图

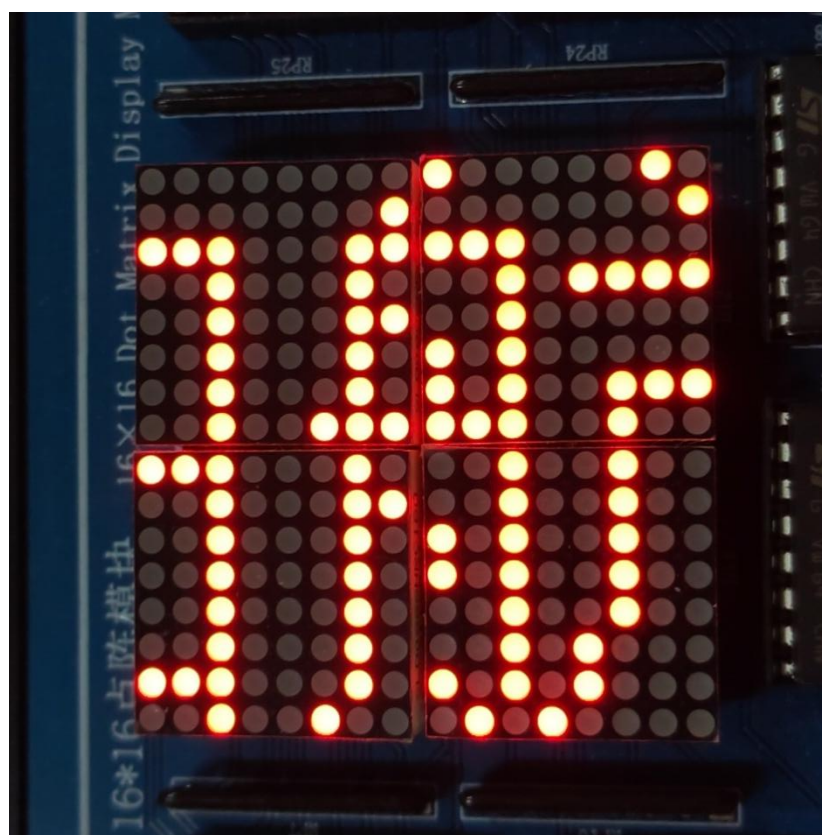


图 4.10 反向滚动最终调试结果图

参考文献

- [1]张海勇,秦艳. 单片机的 16×16 动态扫描 LED 点阵模块设计[J].福建电脑,2025,41(01):37-40.DOI:10.16707/j.cnki.fjpc.2025.01.006.
- [2]赵涟漪. 基于 STC89C52 单片机的 16×16 点阵屏的设计[J].电脑知识与技术,2023,19(22):64-66.DOI:10.14004/j.cnki.ckt.2023.1228.
- [3]张光建. 89S51 单片机的 16×16 点阵汉字显示的设计[J].电子世界,2021,(05):134-136.DOI:10.19353/j.cnki.dzsj.2021.05.054.
- [4]牛晓飞,张秀香,李明,等. 基于 AT89C51 单片机的 16×16 点阵显示屏的设计[J].宿州学院学报,2017,32(12):97-100.DOI:CNKI:SUN:SZSY.0.2017-12-026.
- [5]俞宏霖,李明明,陈旭标,等. 基于 51 单片机的 16×16 LED 滚动式点阵显示研究[J].数码世界,2016,(11):96-98.DOI:CNKI:SUN:SJSM.0.2016-11-071.
- [6]刘硕. 基于单片机的 16×16 LED 点阵屏显示设计[J].才智,2012,(35):42.DOI:CNKI:SUN:CAIZ.0.2012-35-041.
- [7]吕建波. LED 点阵动态显示系统设计及仿真[J].现代电子技术,2012,35(17):154-156.DOI:10.16652/j.issn.1004-373x.2012.17.023.
- [8]同晓荣. 基于 Proteus 的汉字点阵显示设计与仿真[J].信息技术,2012,36(02):33-35+45.DOI:10.13274/j.cnki.hdzj.2012.02.021.
- [9]李皎. 16×16 点阵显示屏的设计[J].科技信息,2011,(32):179+181.DOI:CNKI:SUN:KJXX.0.2011-32-154.
- [10]张琛,耿标. 16×16 点阵 LED 的设计[J].硅谷,2010,(15):64+59.DOI:CNKI:SUN:GGYT.0.2010-15-074.

附录 1 Proteus 仿真电路图

本次课程设计系统主要由单片机最小系统、电源与复位电路、行驱动电路、列驱动电路及按键输入电路五部分组成。

- AT89C51 单片机(U1)是系统的核心控制器。它通过晶振电路提供 11.0592MHz 时钟信号，并由复位电路实现系统初始化。
- 行驱动部分(U5、U7)采用两片 74HC595 串入并出移位寄存器，通过 DS、SH_CP、ST_CP 三根控制线与单片机相连，完成 16 行 LED 信号的并行输出控制。
- 列驱动部分(U4) 使用两片 74HC138 三—八线译码器，实现 16 列信号的逐列选通，节省了单片机 I/O 资源。
- LED 点阵模块 位于电路上方，行端接 74HC595 输出，列端接 74HC138 输出，用于显示汉字或图案。
- 按键模块 (S1-S4) 与单片机 P3.1-P3.4 口相连，实现人机交互功能。

该电路通过行列动态扫描方式实现点阵显示，结构简洁、逻辑清晰，能够稳定完成字符与图文的滚动显示功能，Proteus 仿真电路图如图 5 所示。

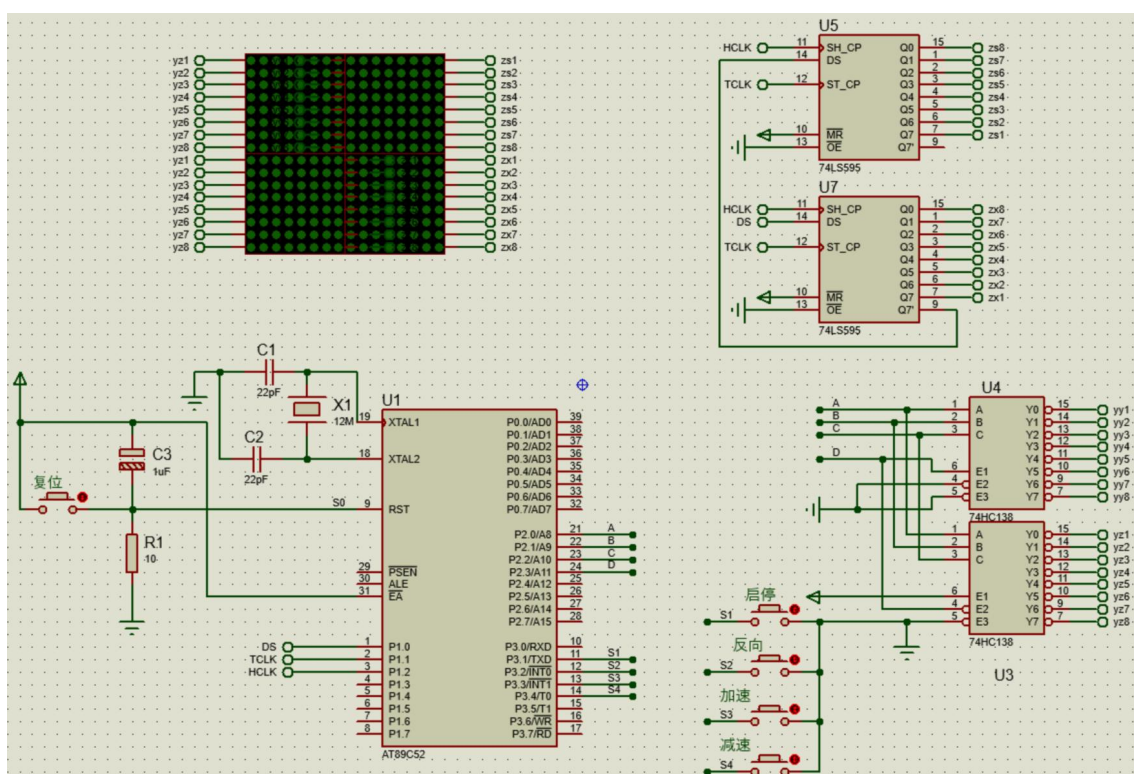


图 5 Proteus 仿真电路图

附录 2 关键部分程序清单

```
#include<reg51.h> // 包含 51 单片机寄存器定义

// 74HC595 控制端口
sbit shcp = P1^2; // 移位时钟 (Shift Clock)
sbit step = P1^1; // 锁存时钟 (Storage Clock / Latch)
sbit ds   = P1^0; // 串行数据输入 (Data Serial)

// 按键输入端口
sbit s1 = P3^1; // 启停按键
sbit s2 = P3^2; // 方向切换按键
sbit s3 = P3^3; // 速度减按键
sbit s4 = P3^4; // 速度加按键

// 全局变量
unsigned char fx; // 滚动方向 (0: 左, 1: 右, 2: 未使用)
unsigned char alt; // 定时器计数变量, 用于控制滚动速度
unsigned int net; // 当前显示的起始字模索引
unsigned char speed=5; // 滚动速度 (1~12)
unsigned char zi=15; // 字符数量
unsigned int zong; // 字符总字节数

// tab、tab1 为点阵字模数组, 每个字符 16 字节 (16x16 点阵)
// tab 为正序, tab1 为逆序或镜像显示, 用于方向切换
unsigned char code tab[]={ /* 点阵数据省略 */
    //字模: 阴码, 16 进制, 顺向, 逐列取
};
unsigned char code tab1[]={
    //字模: 阴码, 16 进制, 顺向, 逐列取
};

// 延时函数
void delay(unsigned int z)
{
    unsigned char x;
    for(; z>0; z--) // 外层循环次数
        for(x=110; x>0; x--); // 内层空循环, 粗略延时
}

// 向 74HC595 写入两字节数据 (16 列)
void writedata(unsigned char dat1, unsigned char dat2)
{

```

```

unsigned char i;
// 写入第 1 字节
for(i=0;i<8;i++)
{
    dat1 = dat1 << 1; // 左移 1 位，最高位送到 CY 标志位
    ds = CY;          // 串行数据输入
    shcp = 1;         // 移位时钟上升沿，74HC595 移入数据
    shcp = 0;
}
// 写入第 2 字节
for(i=0;i<8;i++)
{
    dat2 = dat2 << 1;
    ds = CY;
    shcp = 1;
    shcp = 0;
}
// 锁存数据，使输出口显示
stcp = 0;
stcp = 1;
stcp = 0;
}

// 主程序
void main()
{
    char i, aa;
    TMOD = 0x01; // 定时器 0 工作模式 1 (16 位)
    TH0 = (65536-10000)/256; // 定时器初值高字节
    TL0 = (65536-10000)%256; // 定时器初值低字节
    ET0 = 1;        // 允许定时器 0 中断
    EA = 1;         // 开全局中断
    TR0 = 1;        // 启动定时器 0
    zong = (zi+1)*32; // 计算总字节数 (每字符 16 字节，双字节表示?)

    while(1)
    {
        // 按键检测，启停控制
        if(s1 == 0)
        {
            delay(30); // 按键消抖
            if(s1 == 0)
            {

```

```

        TR0 = ~TR0; // 启停切换
        while(s1 == 0); // 按键松开前等待
    }
}

// 方向切换
if(s2 == 0)
{
    delay(30);
    if(s2 == 0)
    {
        fx++;
        if(fx >= 2) fx = 0; // fx 只循环 0、1
        aa = 0; // 每行显示数据重新计数
        if(fx == 0) net = 0;    // 左滚起点
        if(fx == 1) net = zong; // 右滚起点
        while(s2 == 0);
    }
}

// 速度减按键
if(s3 == 0)
{
    delay(3);
    if(s3 == 0)
    {
        speed--;          // 减速
        if(speed <= 1) speed = 1; // 最小速度限制
    }
    while(!s3);
}

// 速度加按键
if(s4 == 0)
{
    delay(3);
    if(s4 == 0)
    {
        speed++;          // 加速
        if(speed >= 12) speed = 12; // 最大速度限制
    }
    while(!s4);
}

```

```

// 逐行刷新点阵显示
for(i=0;i<16;i++)
{
    P2 = i; // 选择行（列译码？）
    // 根据滚动方向选择数据表
    if(fx == 0) writedata(tab[net+aa], tab[net+aa+1]); // 左滚
    if(fx == 1) writedata(tab1[net+aa], tab1[net+aa+1]); // 右滚
    if(fx == 2) writedata(tab[net+aa], tab[net+aa+1]); // 未使用
    delay(3); // 每行显示延时
    writedata(0,0); // 清空数据，防止残影
    aa += 2; // 每次移动两字节
    if(aa > 30) aa = 0; // 每行最多 32 字节，回到起点
}
}
}

// 定时器 0 中断服务程序，每 10ms 触发一次
void timer0() interrupt 1
{
    TH0 = (65536-10000)/256; // 重新装入初值
    TL0 = (65536-10000)%256;
    alt++; // 定时计数

    // 根据滚动方向调整起点 net
    if(fx == 0 || fx == 1)
    {
        if(alt >= speed) // 根据 speed 控制滚动速度
        {
            alt = 0;
            if(fx == 0) // 左滚
            {
                net += 2;
                if(net > zong) net = 0; // 循环回起点
            }
            if(fx == 1) // 右滚
            {
                net -= 2;
                if(net < 2) net = zong; // 循环回末尾
            }
        }
    }
}
}

```